e-Informatica Software Engineering Journal
Volume 00, Issue 0, pages: 1–6

BibTeX

Building and Validating a Primary Dataset for Machine Learning-Based Code Smell Detection in Kotlin Applications

Argo Wibowo, Adhistya Erna Permanasari*, Teguh Bharata Adji
*Corresponding author: adhistya@ugm.ac.id

Article info

Keywords:
keyword1
keyword2
...

Submitted: 1 Aug 2026
Revised: 1 Sep 2026
Accepted: 1 Oct 2026
Available online: 1 Nov 2026

Abstract

Context: Context ...
Objective: Objective ...
Method: Method ...
Results: Results ...
Conclusions: Conclusions ...

1. Introduction

Software quality is a fundamental aspect of software engineering because it directly influences maintainability, evolvability, and long-term maintenance costs. As software systems continue to grow in size and complexity, maintaining high-quality source code becomes increasingly important to ensure efficient software evolution and reduce technical debt. One of the most widely recognized indicators of poor design quality is code smell, which refers to structural characteristics in source code that suggest potential design deficiencies without necessarily affecting the software's functional correctness. Although code smells do not immediately introduce defects, their accumulation can increase code complexity, reduce readability, complicate testing activities, and ultimately increase maintenance effort.

Various approaches have been proposed to detect code smells, ranging from rule-based and metric-based techniques to more recent machine learning-based methods. Compared with manually defined detection rules, machine learning approaches are capable of learning complex relationships among software metrics and automatically identifying patterns associated with different types of code smells. Consequently, machine learning has become an increasingly popular solution for improving the scalability and adaptability of code smell detection across different software projects.

Recent studies have shown that machine learning has become one of the dominant approaches for code smell detection due to its ability to learn complex relationships among software metrics. Our previous review of code smell detection research over the last decade also indicates a clear shift from traditional rule-based techniques toward machine

learning and optimization-based approaches [1], highlighting the increasing importance of high-quality datasets for model development and evaluation. Furthermore, optimization techniques such as Particle Swarm Optimization combined with Random Forest have demonstrated promising improvements in detection performance, suggesting that advances in learning algorithms should be accompanied by equally reliable datasets to achieve robust and reproducible results [2].

The effectiveness of machine learning models, however, depends heavily on the quality of the datasets used for training and evaluation. A reliable dataset should contain accurate code smell labels, comprehensive software metrics, and a reproducible metric extraction process to ensure consistent results across different studies. In addition, thorough validation of the extracted metrics and dataset characteristics is essential to establish confidence in the resulting models. Without these properties, machine learning experiments may produce unreliable results and become difficult to reproduce or compare with previous studies.

Despite the growing adoption of Kotlin as the preferred programming language for modern Android application development, publicly available datasets specifically designed for Kotlin-based code smell detection remain scarce. Existing datasets are predominantly developed for Java projects and therefore do not adequately represent the characteristics of Kotlin applications. Furthermore, currently available Kotlin datasets generally lack comprehensive classical software metrics, validated Abstract Syntax Tree (AST)-based metric extraction pipelines, and detailed documentation to support reproducibility. Some existing studies also rely on regular-expression-based techniques or parsers with limited support for Kotlin syntax, potentially reducing the accuracy and consistency of the extracted metrics. Consequently, the absence of a validated and reproducible Kotlin software metrics dataset limits the development, evaluation, and fair comparison of machine learning-based code smell detection methods. Although sophisticated learning algorithms have been proposed, their performance remains highly dependent on the quality of the underlying datasets. Existing studies primarily focus on improving classification performance rather than constructing and validating datasets, leaving dataset quality as a relatively underexplored aspect.

To address these limitations, this study constructs a primary software metrics dataset for Kotlin applications using an AST-based metric extraction pipeline implemented with Kopyt. The results obtained from previous study are that the Detekt program can detect various Kotlin-based programming codes well but still has a percentage difference of 48.75% with manual calculations for some advanced metric categories [3]. The proposed pipeline extracts a comprehensive set of classical software metrics, assigns code smell labels, and produces a machine learning-ready dataset. The study further validates the correctness of the extracted metrics, analyzes the statistical characteristics and quality of the dataset, and demonstrates its applicability through baseline machine learning experiments. The remainder of this paper is organized as follows. Section 2 reviews related work on software metrics, code smell detection, and existing datasets. Section 3 describes the dataset construction methodology, including repository selection, metric extraction, and labeling procedures. Section 4 presents comprehensive dataset validation and statistical analyses. Section 5 reports baseline machine learning experiments, while Sections 6 and 7 discuss the findings and threats to validity. Finally, Sections 8 and 9 describe dataset availability and conclude the paper.

This paper makes four main contributions. First, it introduces a primary software metrics dataset for Kotlin applications with code smell labels to support machine learning research. Second, it proposes a reproducible AST-based software metric extraction pipeline

using Kopyt that accurately extracts classical software metrics without relying on regular expressions. Third, it performs comprehensive dataset validation through metric verification, statistical analysis, class distribution analysis, and quality assessment. Finally, it provides baseline machine learning benchmarks that establish the usability of the proposed dataset and facilitate future research on machine learning-based code smell detection for Kotlin applications.

2. Background and Related Work

2.1. Code Smell Detection and Software Metrics

Initially code smell detection was mostly performed using rule-based and metric-based approaches. As software complexity increased, these approaches faced limitations in handling varying project characteristics, leading to the development of machine learning-based methods. Recent research has shown that various machine learning and deep learning algorithms can improve detection accuracy by utilizing a combination of software metrics as predictive features.

2.2. Evolution of Machine Learning-Based Code Smell Detection

Over the past decade, research on code smell detection has shifted from rule-based approaches to machine learning and optimization-based approaches. A previous review study (A Decade of Code Smells Detection: Evolution, Challenges, and Optimization Strategies) showed that improving detection performance increasingly focused on feature selection, hyperparameter optimization, and the use of more sophisticated learning models. However, the study also identified that dataset quality and availability remain key challenges, limiting the reproducibility and comparability of results between studies.

These findings have served as the basis for further research focused on improving detection methods. One such study is the Enhanced Code Smell Detection Using Random Forest Optimized with Particle Swarm Variants study, which integrates feature selection and hyperparameter optimization using various Particle Swarm Optimization (PSO) variants to improve Random Forest performance. The results showed that the combination of feature selection and hyperparameter optimization significantly improved detection accuracy. However, this study used the Fontana dataset, developed for Java projects, and therefore did not adequately represent the characteristics of the Kotlin language.

2.2.1. Kotlin-Based Code Smell Detection

The increasing adoption of Kotlin as the primary language for Android application development has spurred research specifically addressing code smells in Kotlin projects. One previous study, "Custom Rule-Based Feature Engineering for Code Smell Detection in Kotlin Using Detekt," developed a rule-based feature extraction process using Detekt to obtain software characteristics related to code smells. This study demonstrated that Detekt can be used as a starting point for developing features for code smell detection in Kotlin.

However, this research focused on the feature engineering process and did not yet produce a validated software metrics dataset or a reproducible metrics extraction pipeline.

Furthermore, the features obtained relied on Detekt rules and therefore did not directly capture the extraction of classic software metrics through program syntax structure analysis.

These findings indicate that research on code smell detection in Kotlin is still in the development stage of detection methods, while the development of standard datasets usable by the research community is still relatively limited.

2.3. Existing Code Smell Datasets

Most datasets used in code smell detection research are developed using Java projects, such as PROMISE, Qualitas Corpus, and the Fontana Dataset. These datasets have become primary references in the development of various classification methods because they provide software metrics and code smell labels. However, most of these datasets were not designed for Kotlin and therefore lack the ability to adequately represent the syntax and structure of the language.

In addition to the limitations of the programming language, most datasets also lack comprehensive documentation on the metric extraction process or validation of the extraction results. As a result, research reproducibility is limited and the development of new methods requiring high-quality datasets is difficult.

2.4. Research Gap

Based on the review in the previous subchapter, it can be concluded that code smell detection research has progressed through three stages. The first stage focused on the development of rule-based detection methods and software metrics. The next stage focused on utilizing machine learning and model optimization to improve classification performance. More recent research has begun exploring code smell detection in Kotlin, but still focuses on developing detection methods and feature engineering.

However, there is no public dataset available for Kotlin that simultaneously provides comprehensive classical software metrics, a validated and reproducible Abstract Syntax Tree (AST)-based extraction pipeline, comprehensive dataset quality analysis, and initial benchmarks using machine learning. This gap is the primary motivation for this research: to build a primary dataset of software metrics for Kotlin and an AST-based extraction pipeline using Kopyt so that it can serve as a reference in future machine learning-based code smell detection research.

Table 1. Comparison of Existing Code Smell Datasets

Dataset	**Language**	**Classical Metrics**	**Code Smell Labels**	**Public**	**Reproducible Pipeline**
PROMISE	Java	✓	✗	✓	✗
Qualitas Corpus	Java	✓	✓	✓	✗
Fontana	Java	✓	✓	✓	✗
Our Dataset	**Kotlin**	✓	✓	✓	✓

3. Method

Research goal and questions/hypotheses

Research goal …

Research method / procedure

Research method …

4. Results

Results …

5. Discussion

Discussion …

6. Conclusions

Conclusions …

Acknowledgment

CRediT authorship contribution statement

Author 1: Data curation, …. Author 2: Conceptualization, …

Declaration of competing interest

Completing interests

Funding

Founding sources

Declaration of AI assistance in the writing process

During the preparation of this work, the author(s) utilised [*Specify Tool/Service Name*] in order to [*State the Reason for Use, e.g., improve grammar and clarity*]. Subsequent to the application of this tool/service, the author(s) reviewed and edited the content as necessary and accept full responsibility for the final content of the publication.

References

[1] A. Wibowo, A.E. Permanasari, and T.B. Adji, "A decade of code smells detection: Evolution, challenges, and optimization strategies," in *2025 17th International Conference on Information Technology and Electrical Engineering (ICITEE)*, 2025, pp. 1–6.

[2] A. Wibowo, A. Erna, and T. Bharata, "Enhanced code smell detection using random forest optimized with particle swarm variants," *Results in Engineering*, Vol. 29, No. September 2025, 2026, p. 109729.

[3] A. Wibowo, A.E. Permanasari, T.B. Adji, and A. Wibowo, "Custom rule-based feature engineering for code smell detection in kotlin using detekt," in *2025 10th International Conference on Information Technology and Digital Application (ICITDA)*, 2025, pp. 1–8.

Appendix A. Exemplary appendix

Appendix content.

Authors and affiliations

Argo Wibowo
e-mail: argowibowo528951@mail.ugm.ac.id
ORCID: https://orcid.org/0009-0000-2689-5461
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia

Adhistya Erna Permanasari
e-mail: adhistya@ugm.ac.id
ORCID: https://orcid.org/0000-0002-2663-4074
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia

Teguh Bharata Adji
e-mail: adji@ugm.ac.id
ORCID: https://orcid.org/0000-0001-7856-1498
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia